



Blockchain Principles and Applications

Amir Mahdi Sadeghzadeh, PhD

Data and Network Security Lab (DNSL)
Trustworthy and Secure AI Lab (TSAIL)

Layer 2 Scaling: Payment Channels

lightning channels

- make a payment channel bidirectional, and indefinite duration
- but how? refund tx? how to delete / revoke old txs?

timing opcodes

OP_CHECKLOCKTIMEVERIFY

absolute locktime opcode

require that the transaction be confirmed in a block of at least
height n

fail otherwise

timing opcodes

OP_CHECKSEQUENCEVERIFY

relative locktime opcode

require that the input have at least n confirmations to be able to spend

if not, tx fails

revoke based on timing

keyA && keyB ||

keyC && 100 blocks

A and B together can spend any time C can spend together, but must wait A can grab the coins first!

revokable tx

Commit Tx (held by Alice)	
input	output
fund txid Bob's signature	Alice key & 100 blocks or AliceR & Bob key 2 coins
	Bob address 8 coins

13

revokable tx

Commit Tx (held by Bob)	
input	output
fund txid Alice's signature	Alice address 2 coins
	Bob key & 100 blocks or Alice & BobR key 8 coins

reveal to revoke

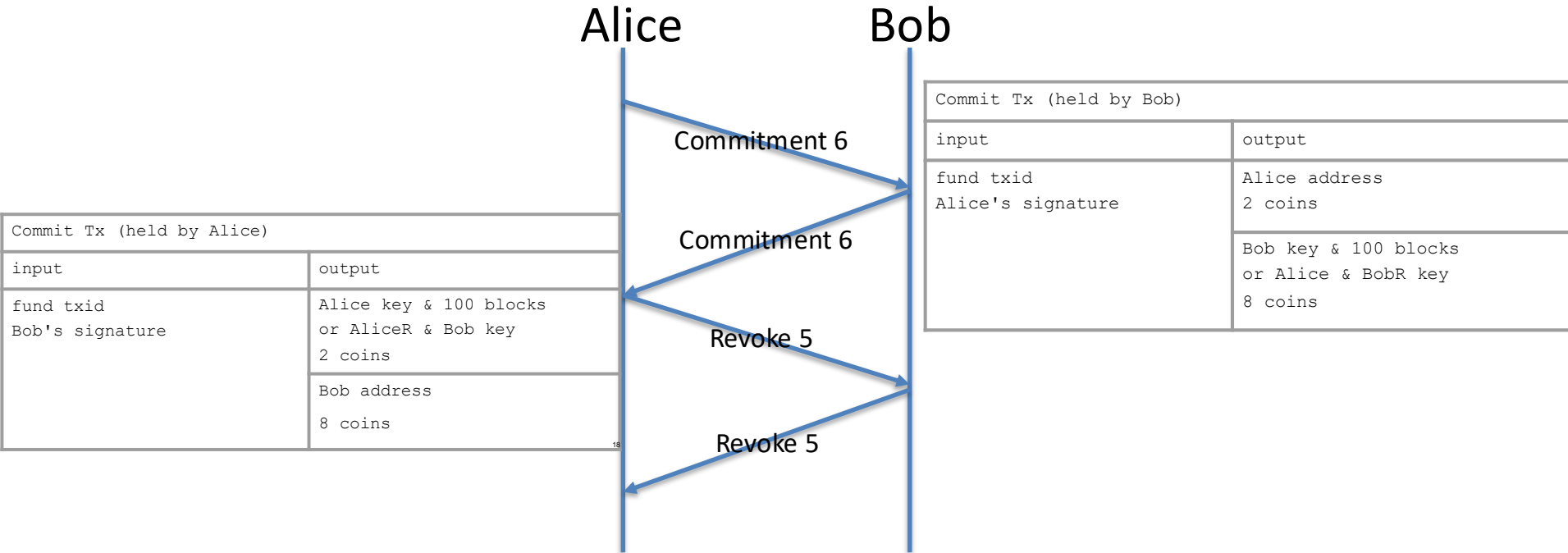
Either party broadcasts & has to wait

Alice gives Bob the AliceR privKey

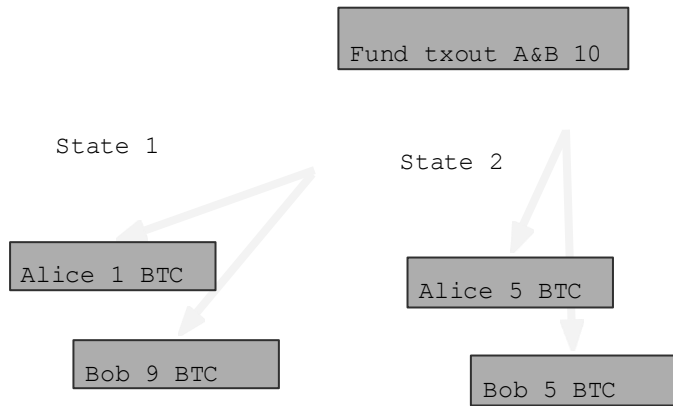
Bob gives Alice the BobR privKey

Now if they broadcast, the counterparty can take all funds
while they wait!

commit state 6 and revoke state 5

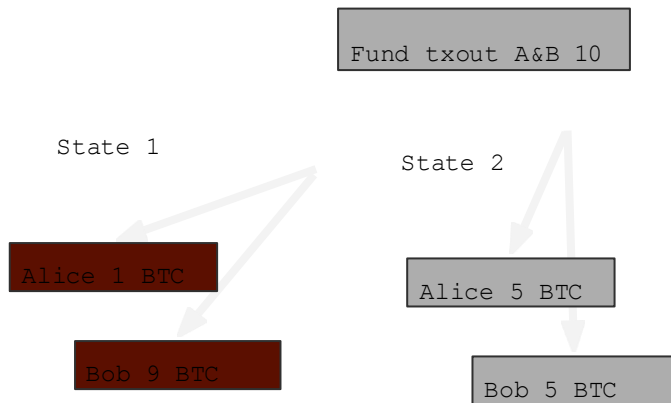


add and delete states



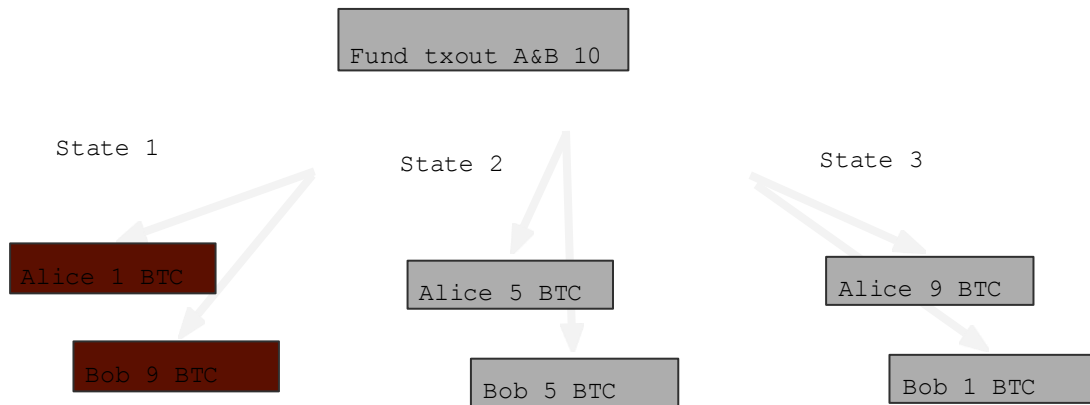
In Lightning, states are added sequentially, and validity is enforced by revealing private keys to previous states

add and delete states



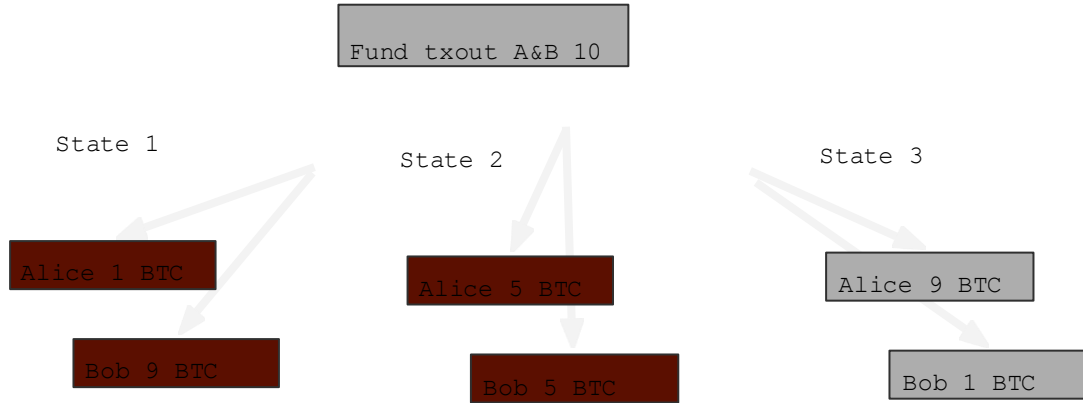
In Lightning, states are added sequentially, and validity is enforced by revealing private keys to previous states

add and delete states



In Lightning, states are added sequentially, and validity is enforced by revealing private keys to previous states

add and delete states



In Lightning, states are added sequentially, and validity is enforced by revealing private keys to previous states

Revocation Keys

- Each side must Keep revocation keys for previous states

2 party, indefinite

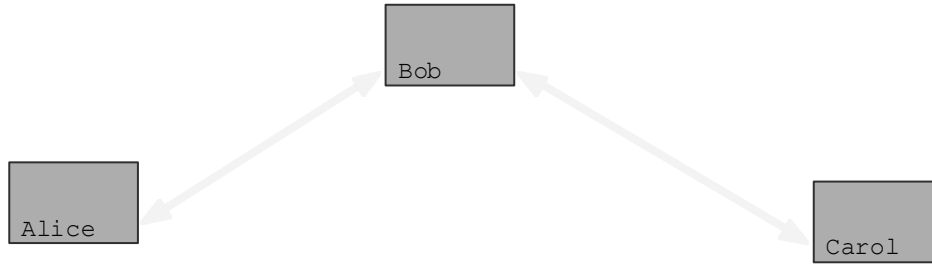
- Still need to create channel to pay
- 1 tx to open, 1 tx to close channel
- potentially 2 txs to close (rare) (broadcast commit tx, sweep)

multiple party channels

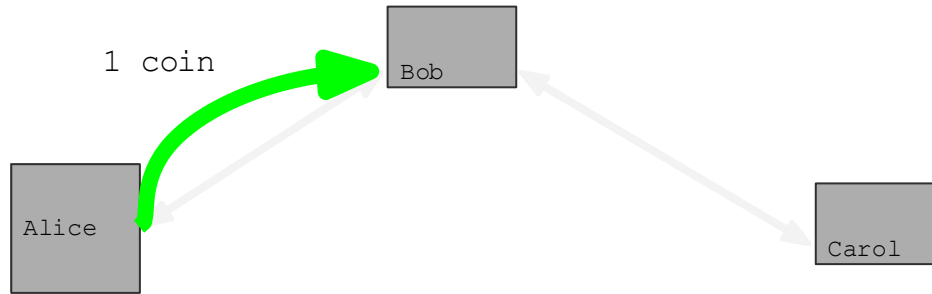
single channel with 3+ users gets really complicated

what about a forwarding network of point to point channels?

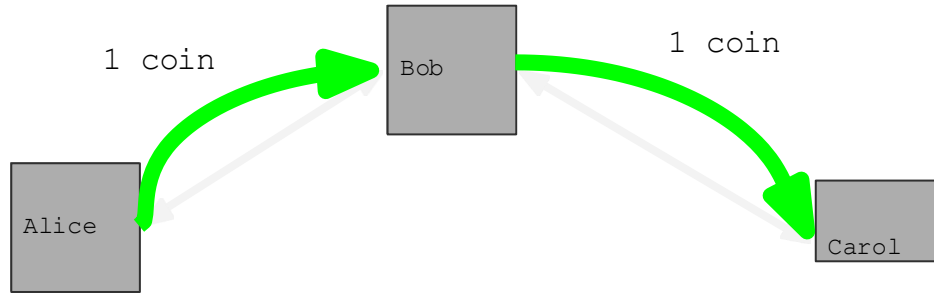
multiple party - optimistic



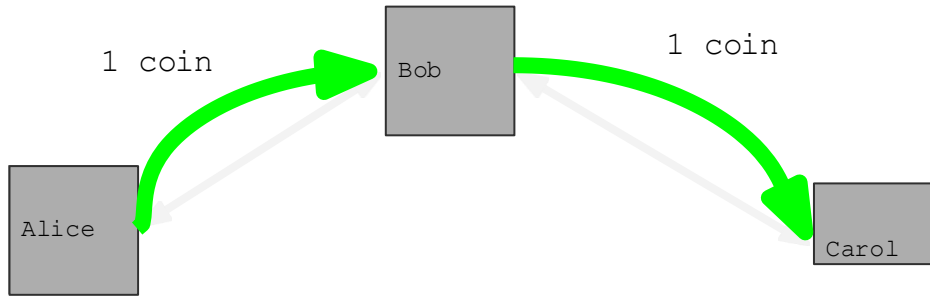
multiple party - optimistic



multiple party - optimistic

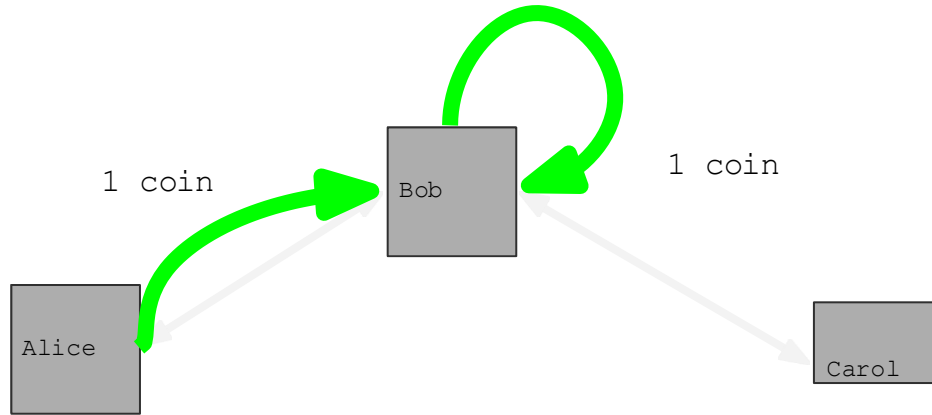


multiple party - optimistic



Alice pays Bob 1 coin, and Bob pays Carol 1 coin

trust issues



Bob keeps the money. Thanks Alice

Preimage determines who spends

New output script type: HTLC

Hash/Time Locked Contract

KeyA && preimageR ||

KeyB && OP_CLTV

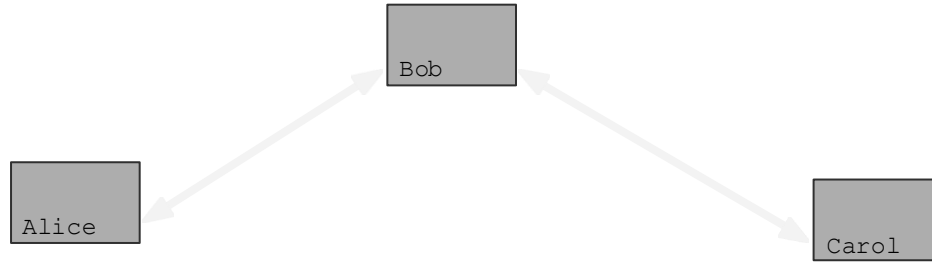
revokable tx

Commit Tx (held by Bob)	
input	output
fund txid Alice's signature	Alice address 2 coins
	Bob key & 100 blocks or Alice & BobR key 8 coins

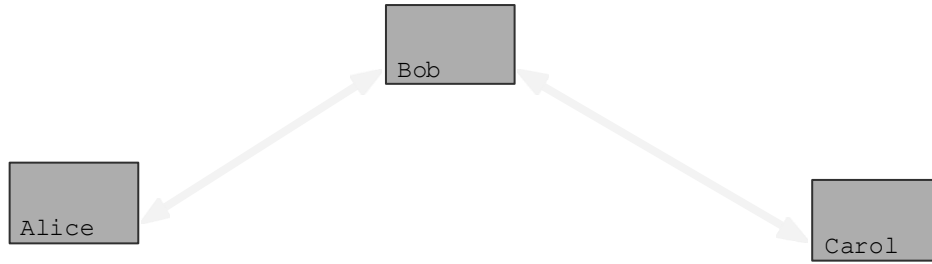
Preimage determines who spends

Commit Tx (held by Bob)	
input	output
fund txid Alice's signature	Alice address: 2 coins
	Bob key && 100 blocks Alice && BobR key 7 coins
	HTLC Alice && R Bob && height 500000 1 coin

multiple party - adversarial

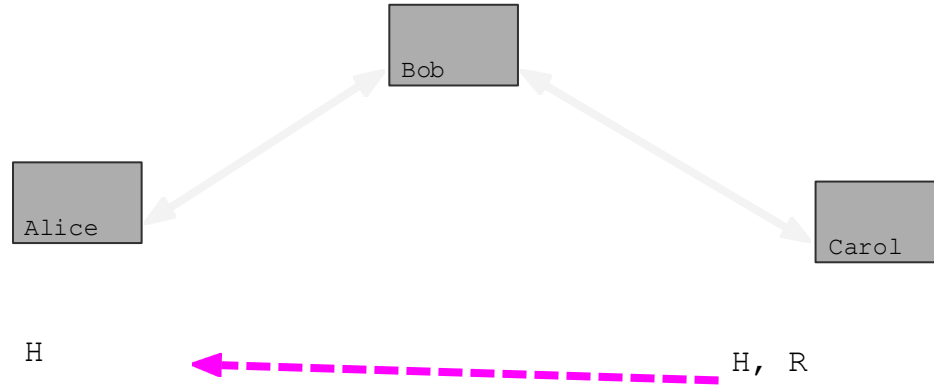


multiple party - adversarial



$$H = \text{hash}(R)$$

multiple party - adversarial

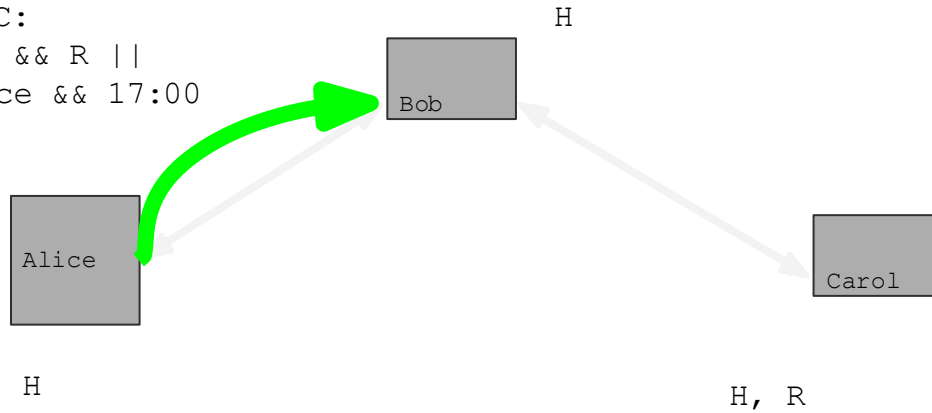


multiple party - adversarial

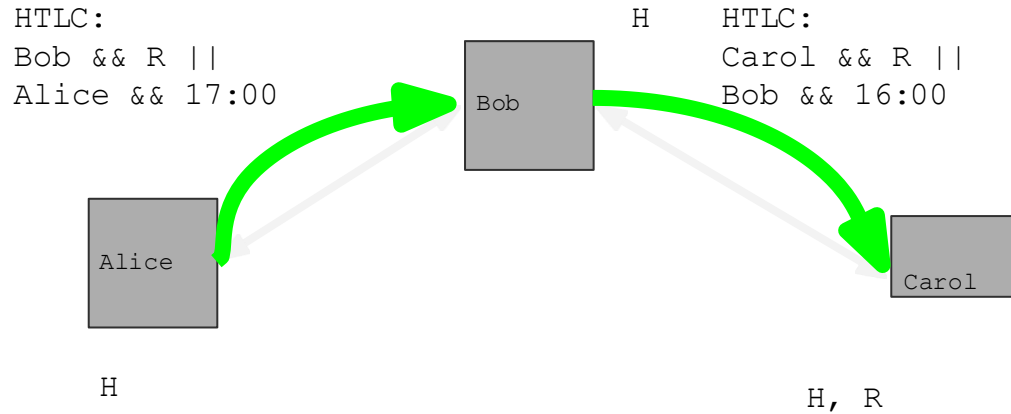
HTLC:

Bob && R ||

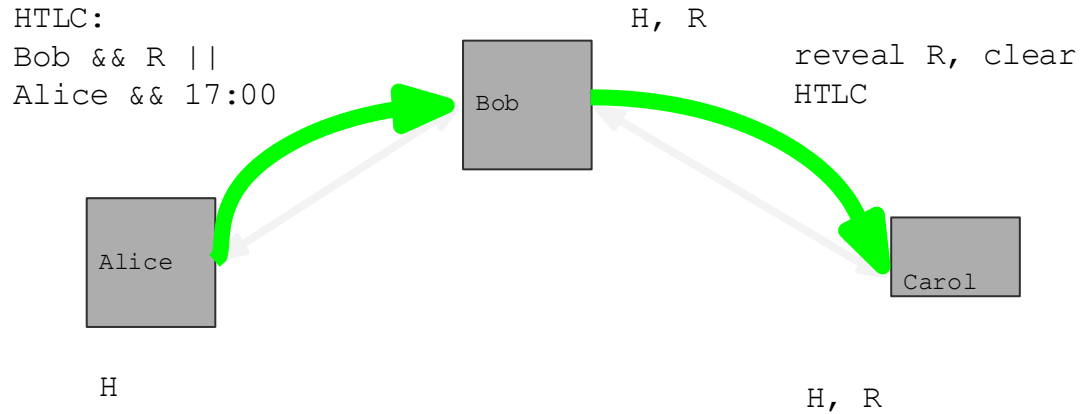
Alice && 17:00



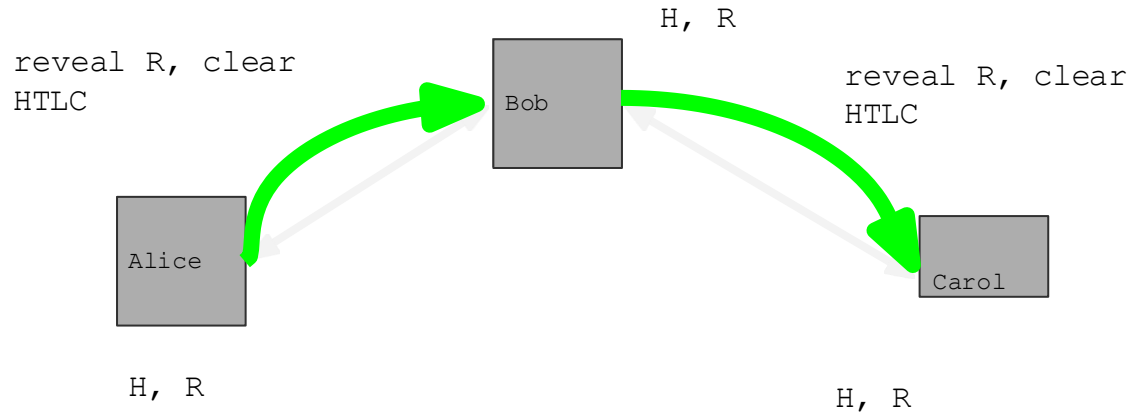
multiple party - adversarial



multiple party - adversarial



multiple party - adversarial



lightning network

- lots of nodes with channels connecting, forming a graph
- request payment routing via HTLC outputs
- open few channels, able to pay many users on the network

Rollups

Scaling the blockchain: the problem

Transaction rates (Tx/sec):

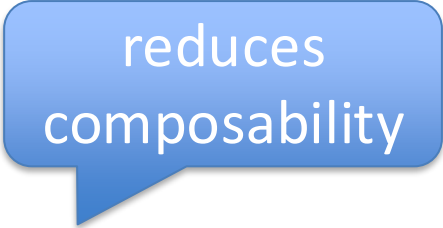
- Bitcoin: can process about **7 (Tx/sec)**
 - Ethereum: can process about **15 (Tx/sec)**
 - The visa network: can process up to **24,000 (Tx/sec)**
- Tx Fees fluctuate:
2\$ to 60\$ for simple Tx

Can we scale blockchains to visa speeds? ... with low Tx fees

How to process more Tx per second

Many ideas:

- Use a faster consensus protocol
- Parallelize: split the chain into independent **shards**
- Today: Rollups, move the work somewhere else
- Payment channels: reduce the need to touch the chain
 - Requires locking up funds; mostly designed for payments.



reduces
composability

Desired properties

- L2 solution
 - Lightweight protocol via smart contracts
 - No change in L1
- Security and data availability
 - Reuse Ethereum security
 - Avoid introducing additional trust
- EVM compatibility
 - Easy to migrate a Dapp from L1 to L2
 - Reduced Tx fees and increased Tx throughput compared to L1

Recall: a basic layer-1 blockchain

Can handle 15 Tx/sec ...



Tx_A



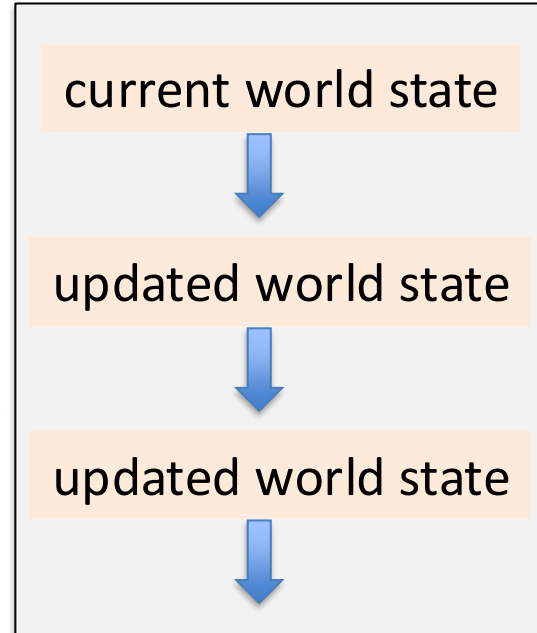
Tx_B



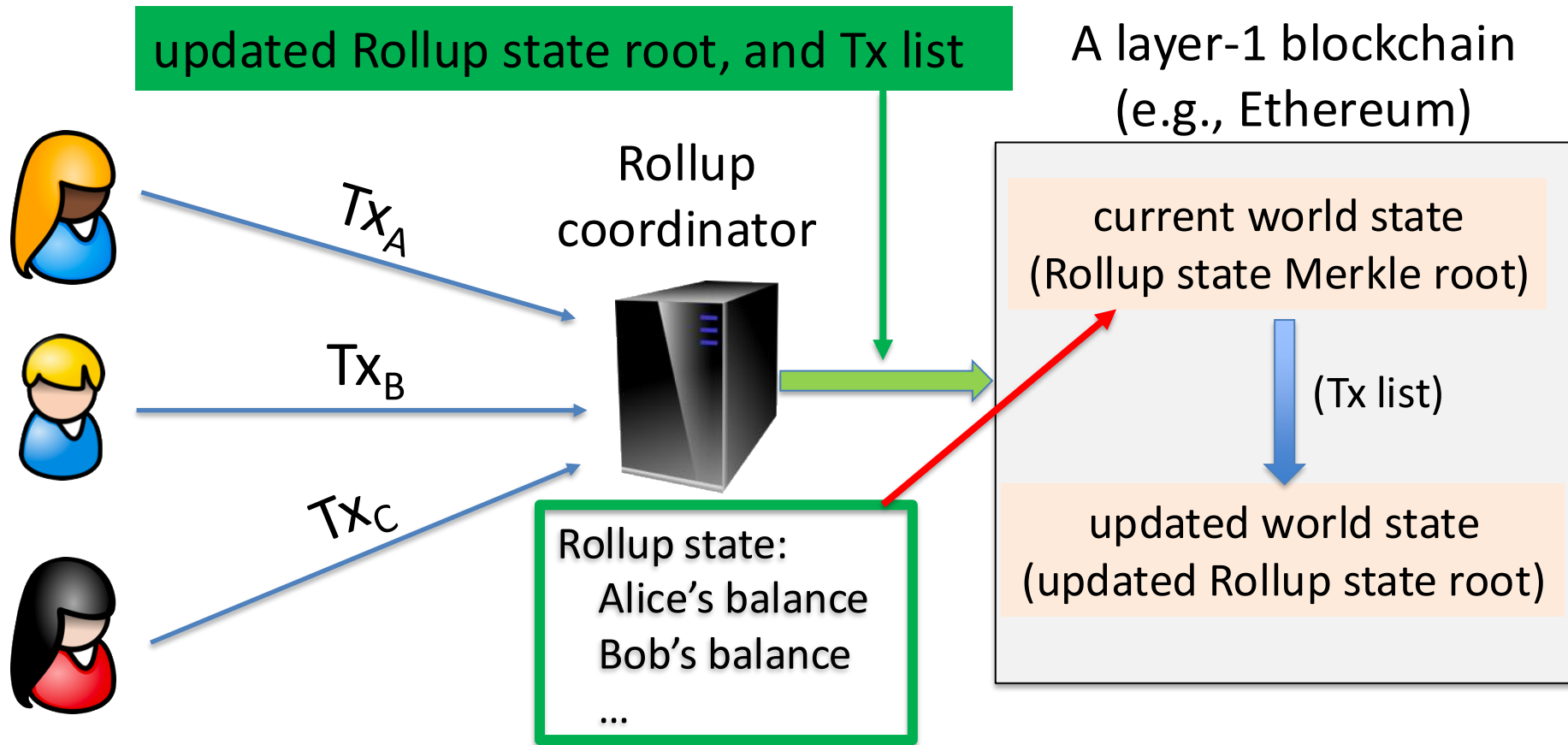
⋮

World state: balances, storage, etc.

A layer-1 blockchain
(e.g., Ethereum)



Rollup idea 1: batch many Tx into one



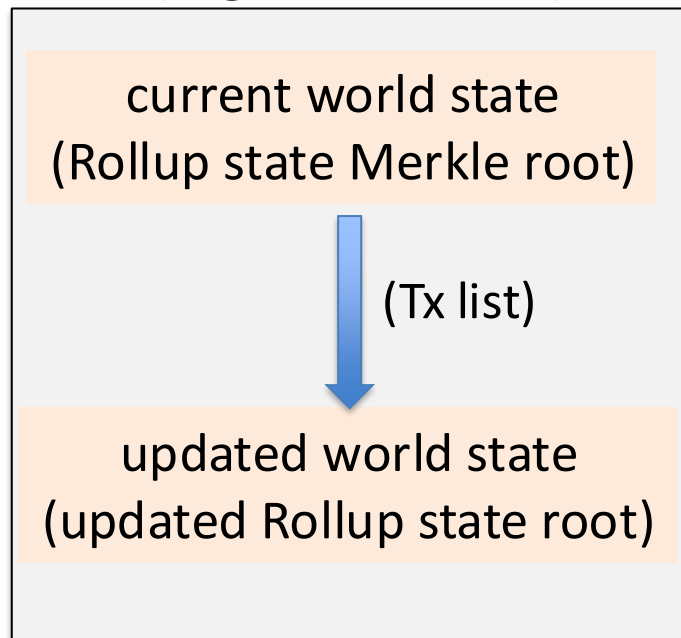
Rollup idea 1: batch many Tx into one

Key point:

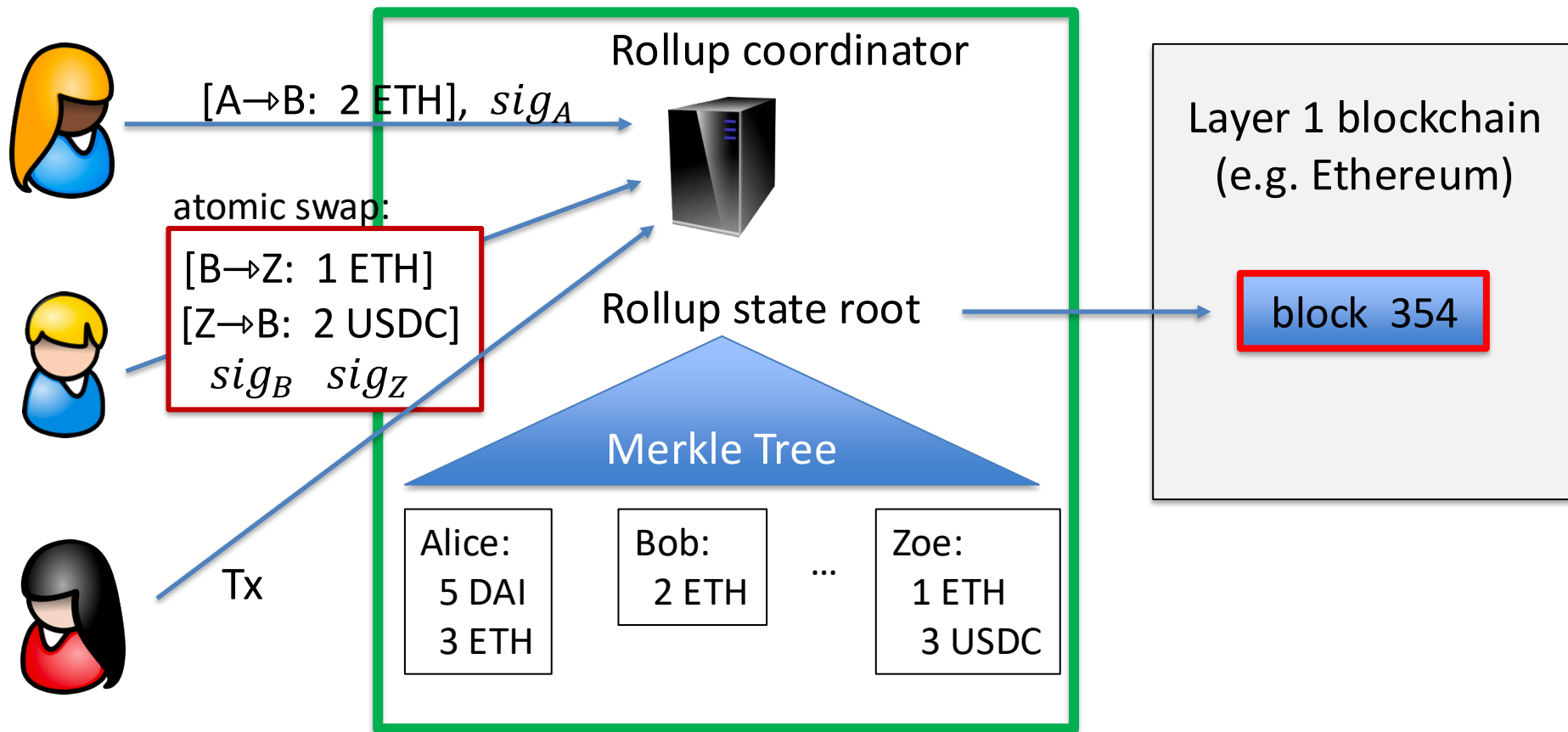
- *Hundreds* of transactions on Rollup state are batched into a *single* transaction on layer-1
⇒ 100x speed up in Tx/sec
- Let's see how ...

Rollup state:
Alice's balance
Bob's balance
...

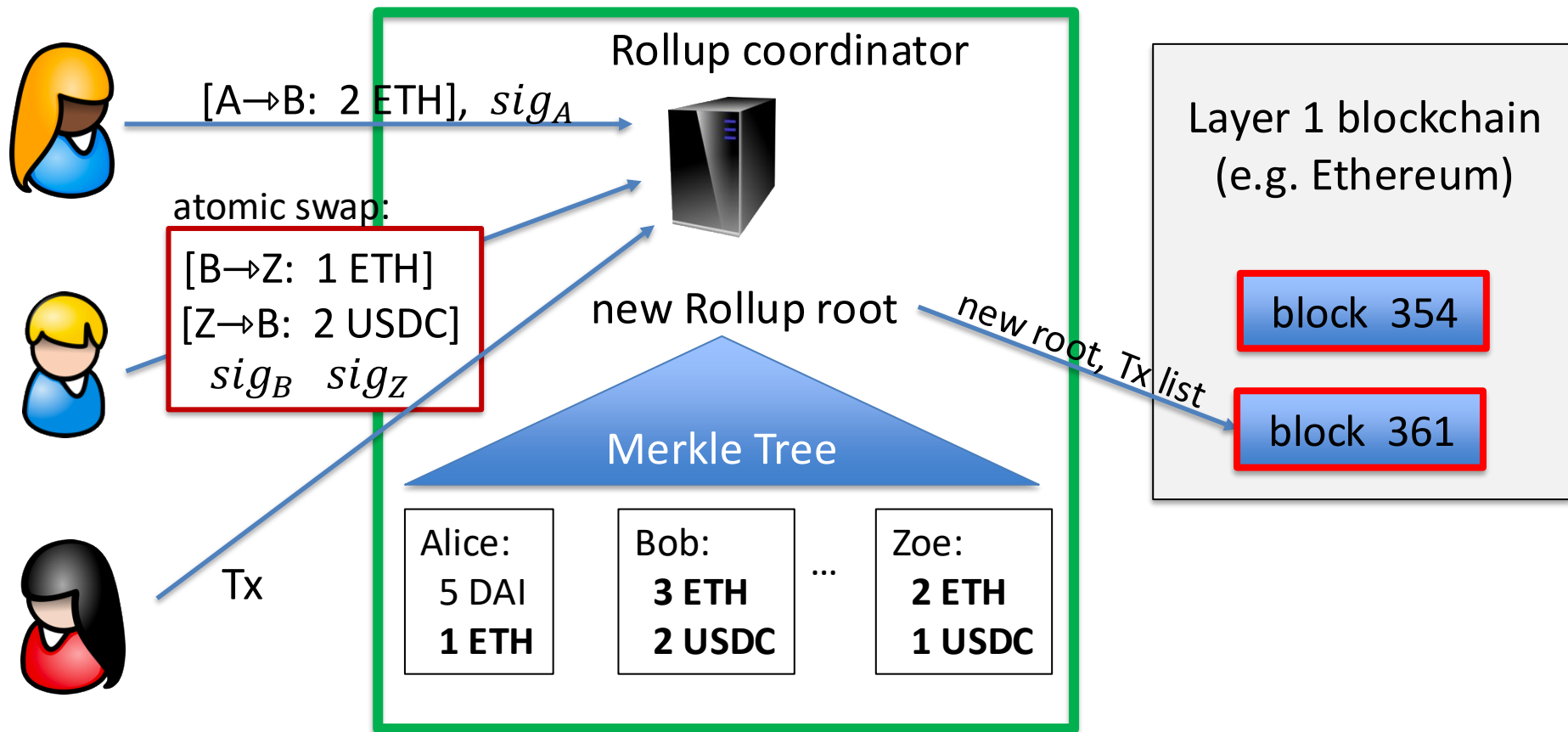
A layer-1 blockchain
(e.g., Ethereum)



Rollup operation (simplified)



Rollup operation (simplified)



In more detail

Rollup contract on
layer-1 holds assets
of all Rollup accounts
(and Merkle state root)

Rollup state (L2)

Alice: 4 ETH, 1 DAI	Bob: 3 ETH, 2 DAI	...
-------------------------------	-----------------------------	-----

(coordinator stores state)

Alice: state	Bob: state	Uniswap: state	Rollup contract: 7 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

Layer-1 blockchain (L1)

Transfers inside Rollup are easy (L2 → L2)



[A→B: 2 ETH], sig_A

(with hundreds of Tx)

Rollup state (L2)

Alice:
4 ETH, 1 DAI

Bob:
3 ETH, 2 DAI

...

Alice:
state

Bob:
state

Uniswap:
state

Rollup contract:
7 ETH, 3 DAI, root

...

Layer-1 blockchain (L1)

Transfers inside Rollup are easy (L2 → L2)

Coordinator updates root on Rollup contract



[A→B: 2 ETH], sig_A

(with hundreds of Tx)

Rollup state (L2)

Alice:
2 ETH, 1 DAI

Bob:
5 ETH, 2 DAI

...

new Merkle root,
Tx list

Alice:
state

Bob:
state

Uniswap:
state

Rollup contract:
7 ETH, 3 DAI, root

...

Layer-1 blockchain (L1)

Transferring funds into Rollup (L1 → L2)

Alice issues an L1 Tx: slow and expensive

Rollup state (L2)



[2 ETH], sig_A

Alice: 2 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	----------------------	-----

Alice: state	Bob: state	Uniswap: state	Rollup contract: 7 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

2 ETH

Layer-1 blockchain (L1)

Transferring funds into Rollup (L1 → L2)

Alice issues an L1 Tx: slow and expensive



[2 ETH], sig_A

Rollup state (L2)

Alice: 4 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	-----------------------------	-----

new Merkle root,
Tx list

Alice: state	Bob: state	Uniswap: state	Rollup contract: 9 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

2 ETH

Layer-1 blockchain (L1)

Transferring funds out of Rollup (L2 → L1)

Requires extra gas on L1 to process transfer



[withdraw 4 ETH], sig_A
(plus hundreds of Tx)

Rollup state

Alice: 4 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	----------------------	-----

Alice: state	Bob: state	Uniswap: state	Rollup contract: 9 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

Layer-1 blockchain (L1)

Transferring funds out of Rollup (L2 → L1)

Requires extra gas on L1 to process transfer



[withdraw 4 ETH], sig_A
(plus hundreds of Tx)

Rollup state

Alice: 0 ETH, 1 DAI	Bob: 5 ETH, 2 DAI	...
-------------------------------	----------------------	-----

new Merkle root,
Tx list

Alice: state	Bob: state	Uniswap: state	Rollup contract: 5 ETH, 3 DAI, root	...
-----------------	---------------	-------------------	---	-----

4 ETH

Layer-1 blockchain (L1)

Summary: transferring Rollup assets

Transactions within a Rollup are easy:

- Batch settlement on L1 network (e.g., Ethereum)

Moving funds into or out of Rollup system ($L1 \Leftrightarrow L2$) is expensive:

- Requires posting more data on L1 network $\Rightarrow$ higher Tx fees.

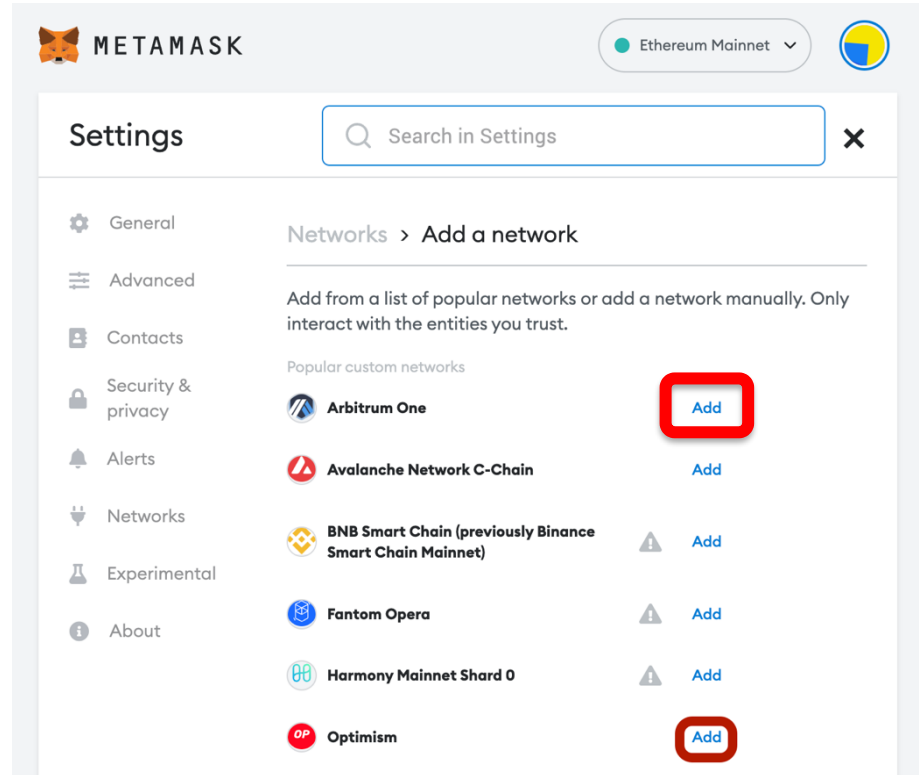
Moving funds from one Rollup system to another ($L2 \Leftrightarrow L2$)

- Either via L1 network (expensive),
or via a direct $L2 \Leftrightarrow L2$ bridge (cheap)

How to send Tx to the coordinator

Enduser configures its wallet to send Tx to the RPC points of the selected Rollup.

(by default Metamask sends Tx to the Ethereum Mainnet RPC points)



The role of the Coordinator

The Coordinator has multiple tasks:

- **Sequence** incoming Tx from Rollup users into a stream of Tx
 - ⇒ can extract MEV from searchers, in addition to Tx fees
 - ⇒ very profitable to be a Rollup coordinator
- **Execute** the stream of Tx on the latest Rollup state
- **Push updates** to the L1 chain

Shared coordinator: one coordinator for multiple Rollups (not today)

Coordinator architectures

A centralized coordinator:

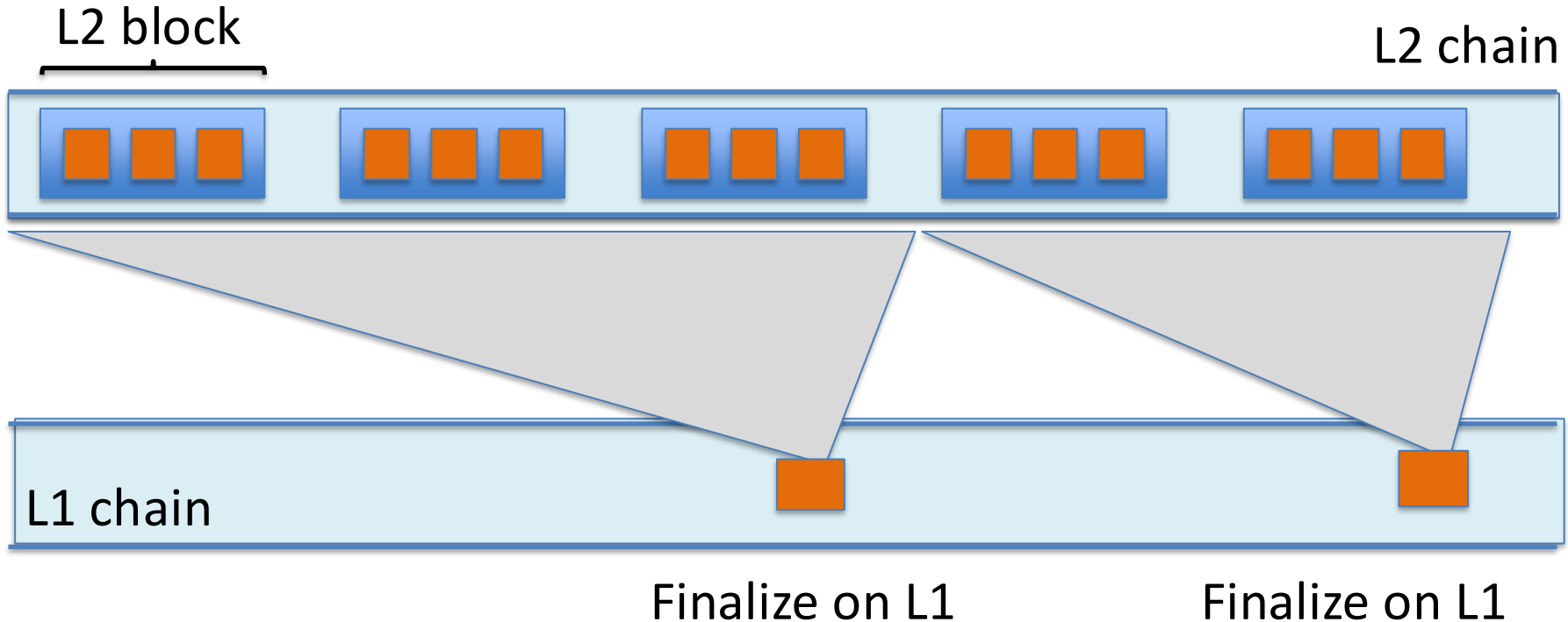
- availability and censorship concerns,
... but cannot steal assets

A decentralized coordinator:

- a set of parties that run a fast consensus protocol
- At every epoch one party is chosen to sequence, execute, and push updates to the L1

Importantly: L1 provides ground truth of the Rollup state

Tx rate on L2 is higher than on L1



An example (StarkNet -- using validity proofs)

Block				
Number ⓘ ⌵	Hash ⓘ	Status ⓘ ⌵	Num. of Txs ⓘ	Age ⓘ ⌵ ⌵
PENDING	PENDING	PENDING	64	3min
13011	0x0432...2380	ACCEPTED_ON_L2	82	8min
13010	0x0492...f0d1	ACCEPTED_ON_L2	122	15min
13009	0x0081...b7af	ACCEPTED_ON_L2	127	24min
		...		
12868	0x060c...15eb	ACCEPTED_ON_L2	58	8h
12867	0x0654...3b0f	ACCEPTED_ON_L1	72	9h
12866	0x0779...57d6	ACCEPTED_ON_L1	63	9h
12865	0x06ae...943f	ACCEPTED_ON_L1	97	9h

Tx posted on L1 (Ethereum) about every eight hours

Source: starkscan.co

Not so simple ...

Problems ...

Problem 1: what if coordinator is dishonest?

- It could steal funds from the Rollup contract
- It could issue fake Tx on behalf of users

Problem 2: what if coordinator stops providing service?

- If Rollup state is lost, how can users issue Tx?
 - ... can't compute updated Rollup Merkle root.

Resources

- ECE/COS 470, Pramod Viswanath, Princeton 2024
- CS251, Dan Boneh, Stanford 2023
- MAS.S62 (Cryptocurrency Engineering and Design), Tadge Dryja, MIT 2018